TECHNICAL ARCHITECTURE & IMPLEMENTATION GUIDE

LogicForge

Technology Stack | Project Structure | 17-Phase Implementation Roadmap

Version 1.0 — Internal Engineering Reference

Confidential — Not for External Distribution

1. Technology Stack Overview

LogicForge is built on a deliberately selected, modern technology stack. Every technology choice is driven by three constraints: suitability for real-time, low-latency evaluation; strong security and sandboxing characteristics; and developer velocity within a monorepo structure. This section documents each technology in the stack, its specific role in the platform, and the reasoning behind its selection.

1.1 Monorepo Tooling — Turborepo

Turborepo is the build orchestration system for the entire LogicForge monorepo. It manages the dependency graph across all packages and applications, enabling incremental builds that skip unchanged workloads. In a platform with multiple independent services and a shared frontend, build times without proper orchestration become a significant developer velocity bottleneck. Turborepo addresses this by caching build outputs both locally and remotely, meaning that a change to the Anti-Cheat service does not trigger a rebuild of the Code Execution Engine or the frontend.

Turborepo also enforces workspace boundaries through its pipeline configuration, ensuring that services only build after their declared dependencies are ready. This is particularly important during CI/CD, where parallel builds must respect the correct execution order.

Capability	How LogicForge Uses It
Incremental Builds	Only services with changed source files are rebuilt; unchanged outputs are restored from cache.
Remote Caching	Build artifacts shared across developer machines and CI runners, eliminating redundant compilation.
Task Pipelines	Enforces correct build order: shared packages build before services that depend on them.
Parallel Execution	Independent services build concurrently, reducing total CI pipeline duration.
Workspace Management	Unified package management across all apps and packages in the repository.

1.2 Containerization — Docker

Every service in LogicForge runs inside a Docker container. Containerization provides the execution environment consistency required for a platform where code submitted by candidates must compile and run identically regardless of the underlying host infrastructure. Beyond environment consistency, Docker enables the independent deployment model that the

LogicForge architecture demands: a deployment to the Anti-Cheat service must not require restarting the Game API or any other service.

The Code Execution Engine specifically depends on Docker's namespace and cgroup isolation capabilities to sandbox candidate code submissions. Submitted Java, C++, and Python programs run inside resource-constrained containers with no network access, strict CPU and memory limits, and read-only filesystem mounts, preventing escape from the sandbox environment.

Service	Container Role
Frontend (Next.js)	Serves the web application and Canvas-rendered challenge UI.
Game API	Manages session state, timer orchestration, and matchmaking.
Question Engine	Handles challenge retrieval, randomization, and semantic variable substitution.
Anti-Cheat Service	Ingests telemetry events and computes session risk scores.
Code Execution Engine	Runs candidate code in nested isolated containers with resource caps.

1.3 Frontend Framework — Next.js

The LogicForge frontend is built with Next.js, React's production framework maintained by Vercel. Next.js is selected for its hybrid rendering model, which allows the platform to server-render static recruiter dashboard pages for fast initial loads while using client-side rendering for the highly interactive game session interfaces. The App Router architecture provides file-based routing with layout nesting, enabling the distinct visual contexts of the lobby, game session, and recruiter dashboard to share authentication state without redundant data fetching.

The Canvas rendering layer sits on top of the Next.js application. Challenges are drawn to HTML5 Canvas elements rather than rendered as DOM content, which is the foundational mechanism of the anti-cheat rendering layer. Next.js's support for Web Workers allows intensive Canvas rendering operations to run off the main thread, preserving UI responsiveness during complex puzzle rendering.

Feature	Usage in LogicForge
App Router	File-system routing for lobby, game session, dashboard, and authentication flows.
Server Components	Recruiter dashboard pages pre-rendered on the server for fast data display.
Client Components	Game session UI, Canvas rendering, WebSocket client management.

API Routes	Thin proxy routes for session initialization and telemetry submission.
Middleware	JWT validation and session guard on protected routes.
Web Workers	Off-thread Canvas rendering for complex puzzle generation without UI jank.

1.4 Backend Services — Node.js with Express

Three of the four backend microservices are built with Node.js and Express: the Game API, the Question Engine, and the Anti-Cheat Service. Node.js is selected for these three services because all three are I/O-bound rather than CPU-bound. The Game API manages WebSocket sessions and timer events. The Question Engine queries a database and applies transformation logic. The Anti-Cheat Service ingests event streams and writes to a risk scoring database. All three patterns play to Node.js's strengths: high-concurrency I/O handling on a non-blocking event loop.

Express is used as the HTTP framework for its minimalism. The services are scoped and focused; they do not benefit from an opinionated full-framework approach. Express provides routing, middleware chaining, and request lifecycle management without imposing unnecessary abstractions.

Service	Why Node.js is Appropriate
Game API	WebSocket session management, event-driven timer logic — highly concurrent, minimal CPU work.
Question Engine	Database queries, JSON transformation, randomization — I/O-bound data pipeline.
Anti-Cheat Service	High-volume telemetry event ingestion, risk score writes — event-stream processing.

1.5 Code Execution Engine — Go

The Code Execution Engine is implemented in Go. This service is the only CPU-bound service in the platform: it must receive code submissions, invoke compilers, execute binaries, capture output, enforce time limits, and return results — all within the tight latency budget required for Arcade Mode evaluation. Go's compiled nature, minimal runtime overhead, and excellent support for subprocess management make it the correct choice for this workload.

Go's goroutines enable the service to handle multiple concurrent compilation and execution requests without the thread-per-request overhead of a traditional server. The standard library's os/exec package provides direct, ergonomic subprocess management for invoking the Java, C++, and Python compilers and runtimes. Go's explicit error handling and lack of a garbage

collection pause during tight loops make it predictably performant under the sustained concurrent load of a live evaluation session.

Requirement	Go Capability
Low-latency execution	Compiled binary, minimal startup overhead, no JIT warm-up cost.
Concurrent job handling	Goroutines handle dozens of simultaneous compile-and-run pipelines.
Subprocess management	os/exec provides direct control over compiler and runtime processes.
Resource enforcement	Integration with Linux cgroups via Docker to cap CPU, memory, and execution time.
Predictable performance	GC pauses are short and bounded; no stop-the-world events during execution.

1.6 Real-Time Communication — WebSockets

WebSockets are used for all real-time bidirectional communication in LogicForge. In Arcade Mode, the timer state, round transitions, and — in Dual Mode — the opponent's live progress must all be pushed to the client without polling latency. A REST polling model would introduce unacceptable jitter into timed evaluation sessions. WebSockets maintain a persistent connection between the game client and the Game API, enabling the server to push timer ticks, round state changes, and matchmaking events to all connected clients with sub-100ms delivery.

The Anti-Cheat Service also uses a WebSocket subscription model to receive telemetry events from the frontend without the overhead of individual HTTP requests per event. High-frequency signals like keystroke cadence and focus events are batched and streamed over a single persistent connection, reducing both server load and network overhead.

1.7 Communication Protocol — REST

REST over HTTPS is used for all non-real-time request-response interactions. Session initialization, question retrieval, result submission, recruiter dashboard data fetching, and user authentication all use REST endpoints. REST is chosen for these flows because they are stateless, infrequent relative to real-time events, and benefit from standard HTTP caching, authentication headers, and status code semantics. The JSON payload format is used uniformly across all REST endpoints.

1.8 Technology Stack Summary

Layer	Technology	Version Target	Purpose
Monorepo Orchestration	Turborepo	Latest Stable	Incremental builds, task pipelines, remote cache
Containerization	Docker + Docker Compose	Latest Stable	Service isolation, sandbox execution, consistent environments
Frontend Framework	Next.js (React)	14.x+	Web UI, Canvas rendering, recruiter dashboards
Game API	Node.js + Express	20.x LTS	Session management, matchmaking, timer orchestration
Question Engine	Node.js + Express	20.x LTS	Challenge retrieval, randomization, variable substitution
Anti-Cheat Service	Node.js + Express	20.x LTS	Telemetry ingestion, behavioral risk scoring
Code Execution Engine	Go	1.22+	Sandboxed compilation and execution for Java, C++, Python
Real-Time Protocol	WebSockets (ws)	Latest Stable	Live session state, timer sync, telemetry streaming
Request Protocol	REST over HTTPS	HTTP/1.1+	Authentication, session init, dashboard data, result submission
Package Manager	pnpm	8.x+	Monorepo workspace dependency management

2. Project Folder & File Structure

LogicForge lives in a single monorepo managed by Turborepo and pnpm workspaces. The repository is organized into two top-level directories: apps, which contains deployable applications, and packages, which contains shared libraries consumed by those applications. This structure enforces clear boundaries between services while enabling code reuse through internal packages.

Convention	All internal packages are prefixed with @logicforge/ in their package.json name field. Cross-service imports use this namespace. No service imports directly from another service's source directory.

2.1 Root Repository Structure

The root of the repository contains workspace configuration, shared tooling configuration, and the Turborepo pipeline definition. No application code lives at the root level.

Path	Type	Description
logicforge/	Root	Monorepo root — workspace configuration lives here
├── apps/	Directory	All deployable applications (frontend + backend services)
├── packages/	Directory	Shared libraries and internal packages
├── turbo.json	Config	Turborepo pipeline definition: build, test, lint, dev tasks
├── pnpm-workspace.yaml	Config	Declares all workspace packages to pnpm
├── package.json	Config	Root package — declares workspaces and shared devDependencies
├── .env.example	Template	Environment variable template — no secrets committed
├── docker-compose.yml	Config	Local development orchestration for all services
├── docker-compose.prod.yml	Config	Production Docker Compose override configuration
└── .gitignore	Config	Repository-wide gitignore rules

2.2 apps/ — Deployable Applications

2.2.1 apps/web — Next.js Frontend

Path	Description
apps/web/	Next.js application root
├── app/	App Router root — all routes live here
│ ├── (auth)/	Route group: login, register, forgot password pages
│ ├── (game)/	Route group: lobby, session, results pages
│ │ ├── lobby/	Pre-session mode and format selection screens
│ │ ├── session/	Active game session — Canvas challenge display
│ │ └── results/	Post-session score summary and breakdown
│ ├── dashboard/	Recruiter dashboard — candidate results, flags, analytics
│ ├── layout.tsx	Root layout — global providers, fonts, metadata
│ └── page.tsx	Landing page
├── components/	Reusable React components
│ ├── canvas/	Canvas rendering components and challenge painters
│ ├── game/	Game-specific UI: timer bar, lives display, round indicator
│ ├── dashboard/	Recruiter-facing tables, charts, and flag displays
│ └── ui/	Generic UI primitives: buttons, modals, loaders
├── hooks/	Custom React hooks: useWebSocket, useSession, useTimer
├── lib/	Client-side utilities: API client, WebSocket manager, auth helpers
├── workers/	Web Workers for off-thread Canvas rendering operations
├── styles/	Global CSS and Tailwind configuration
├── public/	Static assets: pixel-art sprites, fonts, audio
├── next.config.ts	Next.js configuration
├── tailwind.config.ts	Tailwind CSS configuration
├── tsconfig.json	TypeScript compiler configuration
└── Dockerfile	Container definition for production deployment

2.2.2 apps/game-api — Session & Matchmaking Service

Path	Description
apps/game-api/	Game API service root

├── src/	All source files
│ ├── routes/	Express REST route definitions: sessions, matchmaking, health
│ ├── websocket/	WebSocket server setup and event handler registrations
│ ├── handlers/	Business logic: session lifecycle, timer management, dual-mode sync
│ ├── middleware/	Auth validation, request logging, error handling
│ ├── services/	Session store, matchmaking queue, timer engine
│ ├── models/	TypeScript interfaces: Session, Round, Player, MatchmakingEntry
│ ├── config/	Environment config loading and validation
│ └── index.ts	Application entry point — bootstraps HTTP and WebSocket servers
├── tests/	Unit and integration tests
├── tsconfig.json	TypeScript configuration
├── package.json	Service dependencies
└── Dockerfile	Container definition

2.2.3 apps/question-engine — Challenge Delivery Service

Path	Description
apps/question-engine/	Question Engine service root
├── src/	All source files
│ ├── routes/	REST endpoints: question fetch, category list, difficulty tiers
│ ├── handlers/	Request handling: category selection, live mode random dispatch
│ ├── services/	Question retrieval, semantic randomization engine, difficulty scaler
│ ├── randomizer/	Variable name substitution tables and substitution pipeline
│ ├── models/	TypeScript interfaces: Question, Category, Difficulty, SubstitutionMap
│ ├── data/	Seed question definitions (JSON) — source of truth for challenge library
│ ├── config/	Environment config
│ └── index.ts	Entry point

├── tests/	Unit tests covering randomization logic and question integrity
├── tsconfig.json	TypeScript configuration
├── package.json	Service dependencies
└── Dockerfile	Container definition

2.2.4 apps/anti-cheat — Behavioral Telemetry Service

Path	Description
apps/anti-cheat/	Anti-Cheat service root
├── src/	All source files
│ ├── routes/	REST endpoints: flag retrieval, session risk report
│ ├── websocket/	WebSocket server for real-time telemetry ingestion
│ ├── handlers/	Telemetry event processing: focus, keystrokes, timing
│ ├── scoring/	Risk score calculator — applies weighted signal model
│ ├── models/	TypeScript interfaces: TelemetryEvent, RiskScore, SessionFlag
│ ├── storage/	Audit log writer — appends tamper-evident telemetry records
│ ├── config/	Threshold configuration for flag triggers
│ └── index.ts	Entry point
├── tests/	Unit tests for scoring logic and threshold accuracy
├── tsconfig.json	TypeScript configuration
├── package.json	Service dependencies
└── Dockerfile	Container definition

2.2.5 apps/code-runner — Code Execution Engine (Go)

Path	Description
apps/code-runner/	Code Execution Engine root
├── cmd/	Go command entrypoints
│ └── server/	HTTP server entry — starts the execution API
├── internal/	Private application packages (not exported)
│ ├── executor/	Core execution pipeline: receive, compile, run, capture
│ ├── sandbox/	Container lifecycle management for isolated execution

│ ├── languages/	Language-specific compiler and runtime configurations
│ │ ├── java.go	Java compilation and execution settings
│ │ ├── cpp.go	C++ (g++) compilation and execution settings
│ │ └── python.go	Python 3 execution settings (no compilation step)
│ ├── limits/	Resource cap definitions: CPU, memory, wall-clock time
│ └── api/	HTTP handler for submission intake and result return
├── go.mod	Go module definition
├── go.sum	Dependency checksum file
├── Dockerfile	Multi-stage build: Go builder + minimal runtime image
└── Makefile	Build, test, and run shortcuts

2.3 packages/ — Shared Internal Libraries

Package	Name	Description
packages/types/	@logicforge/types	Shared TypeScript type definitions and Zod validation schemas used across all Node.js services and the frontend.
packages/config/	@logicforge/config	Shared environment variable loading utilities and validation logic. All services import from here rather than rolling their own config parsers.
packages/logger/	@logicforge/logger	Structured JSON logger built on Pino. Provides consistent log format, correlation ID injection, and log level configuration across all services.
packages/auth/	@logicforge/auth	JWT token signing, verification, and middleware factory. Services import auth middleware from this package rather than implementing their own.
packages/db/	@logicforge/db	Database client setup and shared query helpers. Provides typed database access functions used by services that require persistence.
packages/eslint-config/	@logicforge/eslint-config	Shared ESLint configuration. All apps and packages extend from this base config to ensure uniform code quality standards.
packages/tsconfig/	@logicforge/tsconfig	Base TypeScript configurations. Services extend the appropriate base (node, nextjs) rather than duplicating tsconfig settings.

3. Implementation Phases — 17-Phase Build Plan

The LogicForge platform is built across 17 sequentially ordered phases. Each phase has a clearly scoped set of deliverables and a completion checklist that must be fully satisfied before the subsequent phase begins. Phases 1 through 3 establish foundational infrastructure. Phases 4 through 7 build all backend services. Phases 8 through 14 build the frontend and game experiences. Phases 15 through 17 deliver the recruiter surfaces, quality validation, and production launch.

Phase	Title	Primary Concern
1	Monorepo Foundation	Turborepo, Docker Compose, workspace setup
2	Shared Packages	Types, config, logger, auth, DB packages
3	Authentication	Registration, login, JWT, role guards
4	Question Engine	Challenge library, randomization, delivery API
5	Code Execution Engine	Go sandbox runner for Java, C++, Python
6	Game API	Session lifecycle, timer, WebSocket events
7	Anti-Cheat Service	Telemetry ingestion, risk scoring, audit log
8	Frontend Foundation	Next.js structure, design system, auth UI
9	Canvas Rendering	HTML5 Canvas renderer, Web Worker, anti-scrape
10	Arcade — Single Player Timer	End-to-end Timer Mode session flow
11	Arcade — Live Mode	Lives system, randomized dispatch, fail state
12	Arcade — Dual Mode	Matchmaking, synchronized sessions, opponent UI
13	Story Mode — The Archive	Database normalization chapter
14	Story Mode — Chapters 2 & 3	OOP/system design and networking chapters
15	Recruiter Dashboard	Results, flags, analytics, live monitoring
16	Testing & Security	Integration, load, penetration, hardening
17	Production Launch	CI/CD, observability, GDPR, launch checklist

PHASE 1

Monorepo Foundation & Developer Environment

Turborepo, pnpm workspaces, Docker Compose, base configurations

Phase 1 establishes the foundational repository structure that every subsequent phase builds upon. The goal is a working monorepo where every developer can clone the repository, run a

single command, and have all services start locally in a consistent environment. No feature work is done in this phase — only infrastructure.

Deliverables & Work Items

- **Initialize pnpm workspace —** Create the root package.json with workspace declarations, pnpm-workspace.yaml, and .npmrc. Verify that pnpm install resolves correctly across an empty workspace.
- **Configure Turborepo —** Create turbo.json defining the initial pipeline tasks: build, dev, test, lint, and type-check. Define input/output file patterns for each task to enable correct cache invalidation.
- **Create all app and package stubs —** Create placeholder package.json and directory structures for all five apps (web, game-api, question-engine, anti-cheat, code-runner) and all seven packages (types, config, logger, auth, db, eslint-config, tsconfig). No implementation code yet — just valid package definitions.
- **Shared TypeScript base configurations —** Implement packages/tsconfig with base.json, node.json, and nextjs.json configurations that all apps will extend.
- **Shared ESLint configuration —** Implement packages/eslint-config with a base config and framework-specific extends for Next.js and Node.js apps. Configure Prettier integration.
- **Docker Compose for local development —** Create docker-compose.yml that starts all five services plus any required infrastructure (database, cache). All services must be reachable on documented local ports. Include a health check for each container.
- **Environment variable templates —** Create .env.example at the root and per-service .env.example files. Document every required and optional variable with type, default, and description. Establish the convention that no secrets are ever committed.
- **Root Makefile or task runner —** Create a root-level Makefile (or equivalent) with targets: install, dev, build, test, lint, docker-up, docker-down. These are the canonical commands documented in the README.
- **README —** Write a developer-facing README covering: prerequisites, local setup steps, service port map, environment variable setup, and how to run individual services in isolation.

Phase Completion Checklist	
☐	pnpm install completes with zero errors across all workspace packages
☐	turbo run build executes successfully on the empty stub workspaces
☐	docker compose up --build starts all five service containers and all health checks pass
☐	All shared packages (@logicforge/types, config, logger, auth, db, eslint-config, tsconfig) exist with valid package.json files
☐	The .env.example file at root is complete and documents every variable
☐	turbo run lint passes with zero errors across all packages

☐	README accurately describes the setup process and a new developer can follow it to a working environment without assistance

PHASE 2

Shared Packages Implementation

@logicforge/types, config, logger, auth, db

Phase 2 implements the internal shared packages that all services depend on. Building these before any service implementation prevents duplicated logic across services and establishes the type contracts that govern cross-service communication. Every package must be fully tested and documented before Phase 3 begins.

Deliverables & Work Items

- **@logicforge/types —** Define TypeScript interfaces and Zod schemas for all domain objects: User, Session, Round, Question, Category, TelemetryEvent, RiskScore, CodeSubmission, ExecutionResult, MatchmakingEntry. Export a validated parsing function for each schema.
- **@logicforge/config —** Implement a typed environment variable loader using a validation library (e.g. Zod or envalid). Provide separate config schemas for each service. Export a getConfig() function that throws a descriptive error on missing required variables, preventing silent misconfiguration.
- **@logicforge/logger —** Implement a structured Pino-based logger factory. The factory must accept a service name and produce a logger that emits JSON with consistent fields: timestamp, level, service, correlationId, message, and an optional data payload. Provide a request-scoped logger middleware for Express services.
- **@logicforge/auth —** Implement JWT signing and verification using a well-audited library. Provide: a sign function that issues access and refresh tokens, a verify function with explicit error types (expired, invalid, missing), and an Express middleware factory that attaches the decoded user payload to the request object. Document the token payload shape.
- **@logicforge/db —** Set up the database client with connection pooling configuration. Implement a typed query helper layer that avoids raw string queries. Define the initial migration files establishing the base schema. Provide a test database utility that rolls back after each test.
- **Unit tests for all packages —** Each package must have a tests directory with coverage of all exported functions. The @logicforge/auth package must test token expiry, invalid signature, and missing token cases. The @logicforge/config package must test missing required variables and type coercion.

Phase Completion Checklist

☐	All five shared packages build without TypeScript errors
☐	turbo run test passes for all shared packages with zero failing tests
☐	@logicforge/types exports validated Zod schemas for every domain object listed in deliverables
☐	@logicforge/config throws a descriptive error when a required environment variable is absent
☐	@logicforge/logger emits JSON with all required fields on every log call
☐	@logicforge/auth correctly rejects expired tokens, tampered tokens, and requests with no Authorization header
☐	@logicforge/db establishes a connection and executes the initial migration without errors
☐	All packages are importable from service packages using their @logicforge/ namespace

PHASE 3

Authentication & User Management

Registration, login, JWT sessions, role-based access

Phase 3 implements the authentication system that controls access to all LogicForge surfaces: the candidate game interface, the recruiter dashboard, and all protected API endpoints. Two roles exist in the system — Candidate and Recruiter — with different permitted actions. This phase delivers the complete auth flow from registration through session refresh.

Deliverables & Work Items

- **User registration endpoint —** Accept email, password, display name, and role. Validate all fields using @logicforge/types schemas. Hash passwords using bcrypt with a minimum cost factor of 12. Store the user record. Return a descriptive error for duplicate email addresses.
- **Login endpoint —** Accept email and password. Validate credentials against the stored hash. On success, issue a short-lived access token (15 minutes) and a long-lived refresh token (7 days). Return both tokens in the response body — never in cookies, which are incompatible with the Canvas-rendering security model.
- **Token refresh endpoint —** Accept a valid refresh token. Verify it against the stored refresh token record. Issue a new access token. Implement refresh token rotation: invalidate the presented token and issue a new refresh token simultaneously.
- **Logout endpoint —** Invalidate the refresh token server-side. The access token is short-lived and relies on its natural expiry; no server-side access token revocation is implemented in the initial release.

- **Auth middleware integration —** Integrate @logicforge/auth middleware into the Game API, Question Engine, and Anti-Cheat services. All protected routes must reject requests with missing, expired, or invalid tokens with a 401 response.
- **Role-based route guards —** Implement a role-check middleware that restricts recruiter-only endpoints (dashboard data, flag reports) to users with the Recruiter role. Candidates attempting to access recruiter routes receive a 403 response.
- **Frontend auth flows —** Implement the login page, registration page, and forgot-password placeholder page in the Next.js app. Implement a client-side auth state manager (React Context or Zustand store) that holds the access token in memory and handles automatic refresh before expiry.

Phase Completion Checklist	
☐	A candidate can register, receive tokens, and access protected candidate routes
☐	A recruiter can register with the recruiter role and access recruiter-only dashboard endpoints
☐	A candidate attempting to access a recruiter route receives a 403 response
☐	An expired access token is rejected with a 401 response from all protected services
☐	Token refresh correctly rotates the refresh token and issues a new access token
☐	Logout invalidates the refresh token and subsequent refresh attempts fail
☐	The frontend auth state manager correctly refreshes the access token before expiry without user intervention
☐	Passwords are never stored in plaintext — only bcrypt hashes are persisted

PHASE 4

Question Engine — Challenge Library & Delivery

Question schema, challenge categories, semantic randomization, REST API

Phase 4 builds the Question Engine — the service responsible for storing, retrieving, and delivering challenges to the game session. This phase includes the core challenge library with at least five questions per category, the semantic randomization pipeline that substitutes variable and function names per session, and the REST API that the Game API queries to obtain challenges.

Deliverables & Work Items

- **Question data schema —** Define the complete JSON schema for a question record. Fields must include: id, category, difficulty tier, problem statement template, answer key,

distractor answers (for multiple-choice modes), code snippet template, substitution slots (which identifiers are subject to randomization), time limit, and point value.

- **Initial question library —** Author a minimum of five questions per category for Timer Mode (The Missing Link, The Bottleneck Breaker, State Tracing) and five additional Syntax Error Detection questions for Live Mode. Questions must cover easy, medium, and hard difficulty tiers within each category. All questions are stored as validated JSON in the apps/question-engine/src/data/ directory.
- **Semantic randomization engine —** Implement the substitution pipeline. Each question defines substitution slots with semantic type annotations (e.g. loop_variable, accumulator, flag_name). The randomizer draws from category-appropriate name pools, applies substitutions consistently within a question (the same slot always gets the same replacement within one question instance), and returns the transformed question. The transformation must be deterministic given a fixed seed, enabling audit replay.
- **Question retrieval REST API —** Implement endpoints: GET /categories (list available categories with metadata), GET /question (accept category, difficulty, session_id; return a randomized question instance), GET /question/:id/answer (accept session_id and answer; return correctness and explanation — used after round completion).
- **Difficulty scaling —** Implement a simple difficulty scaler: the first two rounds of a session draw from easy questions, the next two from medium, and the final round from hard. This logic is configurable via the service config, not hardcoded in the route handler.
- **Question deduplication within session —** The service must track which question IDs have been served within a session (using the session_id parameter) and never serve the same question twice in the same session. Implement this as an in-memory set per session with a configurable TTL.

Phase Completion Checklist	
☐	GET /categories returns the correct category list with accurate metadata
☐	GET /question returns a valid, schema-conforming question with substituted variable names
☐	The same question requested with the same session_id always returns the same substitution mapping (deterministic randomization)
☐	Two requests with different session_ids for the same question return different variable name substitutions
☐	No question is served twice within the same session
☐	The difficulty scaler returns easy questions for rounds 1-2, medium for rounds 3-4, and hard for round 5
☐	All five Live Mode Syntax Error Detection questions are present and correctly schema-validated
☐	The question library contains at minimum five questions per category across three difficulty tiers

PHASE 5

Code Execution Engine — Sandboxed Runner

Go service, Java / C++ / Python compilation, resource limits, execution API

Phase 5 builds the most security-critical service in LogicForge: the Code Execution Engine. This Go service accepts code submissions, compiles them (where applicable), executes them in an isolated environment with strict resource limits, and returns the result. Correctness and security are equally important in this phase — a misconfigured sandbox is a platform-wide security vulnerability.

Deliverables & Work Items

- **Go HTTP server —** Initialize the Go module and implement the HTTP server using the standard net/http package. Expose a single submission intake endpoint: POST /execute. Implement a health endpoint: GET /health.
- **Language execution configurations —** Implement separate configuration structs for Java, C++, and Python. Each configuration specifies: the compiler binary path (javac, g++), the runtime binary path (java, the compiled binary, python3), required compiler flags, the file extension for the source file, and the execution command template.
- **Compilation pipeline —** For Java and C++: write the submitted source to a temporary file in a per-submission scratch directory, invoke the compiler, capture stdout and stderr, detect compilation failure by exit code, and return a CompilationError result if compilation fails. For Python: skip compilation and proceed directly to execution.
- **Execution sandbox —** Execute the compiled binary (or Python script) inside a Docker container with: no network access (--network=none), read-only filesystem except for the scratch directory, CPU limit (0.5 cores), memory limit (128MB), and a wall-clock timeout (10 seconds for Arcade Mode, 30 seconds for Story Mode). Kill the container on timeout. Return a TimeLimitExceeded result.
- **Output capture and result schema —** Capture stdout (up to 4KB), stderr (up to 1KB), and exit code from the execution container. Return a JSON result with fields: status (success, compilation_error, runtime_error, time_limit_exceeded), stdout, stderr, execution_time_ms, and memory_used_bytes.
- **Concurrent submission handling —** Use goroutines to handle multiple concurrent submissions. Implement a configurable submission pool with a maximum concurrency limit. Submissions beyond the pool limit receive a 429 response with a Retry-After header.
- **Security hardening —** Verify that submitted code cannot: write to paths outside the scratch directory, make network connections, execute shell commands that spawn processes outside the sandbox, or consume more than the declared resource limits. Document the threat model and any known limitations.

Phase Completion Checklist	
☐	A valid Java submission compiles and returns the correct stdout

☐	A valid C++ submission compiles and returns the correct stdout
☐	A valid Python 3 submission executes and returns the correct stdout
☐	A Java submission with a syntax error returns status=compilation_error with the compiler error message in stderr
☐	A submission that runs an infinite loop is killed after the configured timeout and returns status=time_limit_exceeded
☐	A submission that attempts a network connection fails with a connection refused error (network isolation confirmed)
☐	A submission that attempts to write outside the scratch directory fails with a permissions error
☐	Ten concurrent submissions are all handled within 2x the single-submission latency (concurrency confirmed)
☐	GET /health returns 200 when the service is running

PHASE 6

Game API — Session Lifecycle & Timer Engine

Session creation, round management, timer orchestration, REST + WebSocket

Phase 6 builds the Game API, the orchestration hub of every game session. This service creates sessions, manages round transitions, enforces timers, and serves as the real-time event bus for the frontend. At the end of this phase, a session can be created, progressed through five rounds, and concluded — though the frontend is not yet connected.

Deliverables & Work Items

- **Session creation endpoint —** POST /sessions — accept mode (single, dual), session_type (timer, live), and category (required for timer mode, absent for live mode). Validate inputs, create a session record, fetch the first question from the Question Engine, and return the session_id and first question. Assign a unique, unguessable session token that the frontend uses for WebSocket authentication.
- **Session state machine —** Implement the session state machine with states: LOBBY, ROUND_ACTIVE, ROUND_COMPLETE, SESSION_COMPLETE, SESSION_FAILED (lives exhausted). All state transitions are logged with timestamps. Invalid transitions (e.g. submitting an answer when the session is in LOBBY state) return a 409 Conflict response.
- **Timer engine —** Implement a per-round countdown timer. When a round starts, set a timer for the question's configured time limit. On expiry, the session transitions to ROUND_COMPLETE with a TIMEOUT result. The timer must be cancellable when a valid answer is submitted before expiry. All timer events are broadcast over WebSocket to the connected client.

- **Answer submission endpoint —** POST /sessions/:id/answer — accept the session_id and the candidate's answer. Validate the session state. Cancel the active timer. Query the Question Engine for the correct answer. Record the round result. If lives are active (Live Mode), deduct a life on incorrect answer. Transition state to ROUND_COMPLETE and broadcast the result over WebSocket.
- **Round advance endpoint —** POST /sessions/:id/next-round — advance to the next round, fetch the next question from the Question Engine, reset the timer, and broadcast the new round state over WebSocket. If the session has completed all five rounds, transition to SESSION_COMPLETE.
- **WebSocket event broadcasting —** Establish a WebSocket server. When a client connects, authenticate using the session token. Subscribe the connection to the session's event channel. Broadcast the following events: TIMER_TICK (every second), ROUND_START, ROUND_COMPLETE, SESSION_COMPLETE, LIFE_LOST (Live Mode), SESSION_FAILED.
- **Session result endpoint —** GET /sessions/:id/result — return the complete session result: all rounds with answers, scores, time-per-round, total score, and outcome. This is the payload used by the frontend results page and the recruiter dashboard.

Phase Completion Checklist	
☐	POST /sessions creates a session and returns a valid session_id and first question
☐	A TIMER_TICK WebSocket event is broadcast every second from round start
☐	An answer submitted after the timer expires returns a 409 response (timer already concluded the round)
☐	A correct answer cancels the active timer and broadcasts ROUND_COMPLETE with outcome=CORRECT
☐	An incorrect answer in Live Mode broadcasts LIFE_LOST and decrements the life count
☐	After five rounds, the session transitions to SESSION_COMPLETE and no further submissions are accepted
☐	Lives exhausted in Live Mode transitions the session to SESSION_FAILED
☐	GET /sessions/:id/result returns complete round-by-round data for a completed session
☐	An unauthenticated WebSocket connection attempt is rejected

PHASE 7

Anti-Cheat Service — Telemetry & Risk Scoring

Behavioral event ingestion, signal processing, risk score model, audit log

Phase 7 builds the Anti-Cheat Service. This service receives a continuous stream of behavioral telemetry events from the frontend during every session and processes them into a real-time risk score. The risk score is visible to recruiters after the session and is updated throughout the session. All telemetry events are written to an append-only audit log for later review.

Deliverables & Work Items

- **WebSocket telemetry ingestion —** Establish a WebSocket endpoint that the frontend connects to at session start. Accept a stream of typed telemetry events. Event types: FOCUS_LOST, FOCUS_RESTORED, KEYSTROKE_BURST (payload: character count and duration), PASTE_DETECTED, MOUSE_INACTIVE (duration), SOLUTION_SUBMITTED (payload: time-to-solution in ms, round number).
- **Risk scoring model —** Implement a weighted additive risk scoring model. Each event type contributes a configurable number of risk points: FOCUS_LOST adds 10 points per occurrence, PASTE_DETECTED adds 25 points, implausibly fast SOLUTION_SUBMITTED (below the configured baseline for the question difficulty) adds 30 points. The score is capped at 100. Thresholds are defined in the service config, not hardcoded.
- **Baseline calibration data —** Define per-difficulty baseline solution times based on the question library. These baselines are used to determine whether a solution time is implausibly fast. Document the methodology: baselines are the 10th percentile of expected human solution times for each difficulty tier, derived from the question design process.
- **Real-time risk score broadcast —** After each scoring event, compute the updated risk score and broadcast it to a recruiter WebSocket subscription for the session. Recruiters monitoring a live Dual Mode session see the risk score update in real time.
- **Flag generation —** When the risk score crosses 60, generate a MEDIUM flag. When it crosses 80, upgrade to a HIGH flag. Store flags with: session_id, timestamp, flag level, contributing events, and current score. Flags are never downgraded within a session — only upgraded.
- **Audit log —** Write every telemetry event to an append-only audit log. Each log entry contains: event_type, session_id, candidate_id, timestamp (server-side, not client-supplied), and event payload. The audit log must not be mutable after write. Implement as an append-only database table with no UPDATE or DELETE permissions granted to the service's database user.
- **Flag retrieval REST API —** GET /sessions/:id/flags — return the flag list and current risk score for a session. GET /flags?recruiter_id=X — return all flagged sessions across a recruiter's candidate pool, sorted by risk score descending.

Phase Completion Checklist	
☐	A FOCUS_LOST event correctly increments the risk score by the configured weight
☐	A PASTE_DETECTED event correctly increments the risk score by the configured weight
☐	A solution submitted below the difficulty baseline correctly increments the risk score

☐	A risk score crossing 60 generates a MEDIUM flag persisted to the database
☐	A risk score crossing 80 upgrades the flag to HIGH
☐	All telemetry events are written to the audit log with server-side timestamps
☐	The audit log table has no UPDATE or DELETE privileges for the service database user (confirmed via database permission inspection)
☐	GET /sessions/:id/flags returns the correct flag list and current risk score
☐	An unauthenticated telemetry WebSocket connection is rejected

PHASE 8

Frontend Foundation — Next.js App Structure & Design System

Routing, layout system, Tailwind design tokens, pixel-art aesthetic, auth flows

Phase 8 establishes the frontend application structure and implements the visual design system that all subsequent UI phases build upon. This phase delivers working authentication pages and the navigation shell, but no game session UI. The pixel-art aesthetic and design token system are finalized here so that all subsequent UI work is consistent.

Deliverables & Work Items

- **App Router route structure —** Create all route directories and placeholder page.tsx files: (auth)/login, (auth)/register, (game)/lobby, (game)/session, (game)/results, and dashboard. Each page renders a titled placeholder confirming the route is reachable.
- **Root layout and providers —** Implement the root layout with: global font loading (pixel-art display font for headings, clean sans-serif for body), auth state provider, WebSocket manager provider, and React Query (or equivalent) client provider for server data fetching.
- **Tailwind design token system —** Define all design tokens in tailwind.config.ts: a primary color palette (deep navy, arcade blue, signal green, warning amber, danger red), a pixel-art font scale, spacing scale, and a border-radius scale that defaults to sharp edges (0 or 2px radius) to reinforce the retro aesthetic.
- **Core UI component library —** Implement in components/ui/: Button (variants: primary, secondary, danger, ghost), Input (with error state), Modal (with focus trap), Loader (pixel-art spinner animation), Badge (for difficulty, flag level), and Tooltip.
- **Login and registration pages —** Implement fully functional login and registration pages using the core UI components. Connect to the auth API. On successful login, store the access token in the auth state manager and redirect to /lobby. Display server-side validation errors inline.

- **Navigation shell —** Implement a top navigation bar visible on all authenticated pages. Show the user's display name, role badge, and a logout button. The nav bar is rendered as a Server Component with minimal client hydration.
- **Protected route middleware —** Implement Next.js middleware that redirects unauthenticated users to /login for all routes under (game)/ and dashboard/. Redirect authenticated users away from (auth)/ routes.

Phase Completion Checklist	
☐	All route placeholders are reachable at their documented paths without 404 errors
☐	Login with valid credentials stores the access token and redirects to /lobby
☐	Login with invalid credentials displays a descriptive inline error without page reload
☐	An unauthenticated request to /lobby redirects to /login
☐	An authenticated user visiting /login is redirected to /lobby
☐	All Tailwind design tokens are defined and the pixel-art color palette is applied consistently
☐	All six core UI components render correctly and handle their documented props and states
☐	The navigation bar displays the correct user display name and role badge after login

PHASE 9

Anti-Cheat Rendering — Canvas Challenge Display

HTML5 Canvas renderer, challenge painting, Web Worker integration, copy-paste prevention

Phase 9 implements the Canvas rendering layer — the foundation of LogicForge's anti-cheat architecture. All challenges must be displayed exclusively as Canvas-rendered pixels. No challenge content may exist as readable DOM text nodes at any point during an active session. This phase delivers the Canvas rendering engine and its Web Worker integration.

Deliverables & Work Items

- **Canvas renderer architecture —** Design the Canvas renderer as a class that accepts a question object and a Canvas element reference and paints the challenge UI. The renderer is responsible for: background rendering, code block display with syntax highlighting applied as pixel colors (not DOM spans), question text rendering, and answer input zone painting.
- **Web Worker integration —** Move all Canvas rendering operations to a Web Worker using an OffscreenCanvas. The main thread transfers the OffscreenCanvas to the

worker at session start. The worker receives paint commands via postMessage and renders to the offscreen canvas without blocking the main thread. This prevents UI jank during complex challenge rendering.

- **Monospace code block renderer —** Implement a pixel-accurate monospace code block renderer on the Canvas. Each character is rendered individually from a pre-loaded bitmap font sprite sheet. The renderer supports: syntax token coloring (keywords, identifiers, operators, literals), line numbering, highlighted missing-code zones (for The Missing Link challenges), and horizontal scrolling for long lines.
- **Answer input zone —** Render an answer input zone on the Canvas for challenges that require text entry. The input zone captures keyboard events directly on the Canvas element and maintains an internal cursor and text buffer. The text buffer is never reflected in a DOM input element — it exists only in JavaScript memory and is rendered as pixels.
- **Copy-paste prevention —** Disable the contextmenu, copy, cut, and selectstart events on the Canvas element and its containing document during an active session. Verify that attempting to copy the challenge content via keyboard shortcut, right-click menu, or browser developer tools selection yields no usable text.
- **Accessibility considerations —** Document the deliberate accessibility trade-offs of Canvas rendering. Implement a recruiter-controlled accessibility mode (enabled per-session by the recruiter before starting) that disables Canvas rendering and falls back to standard DOM display. This mode is logged and its activation is visible to the recruiter. This mode is explicitly excluded from the anti-cheat telemetry scoring.

Phase Completion Checklist	
☐	A challenge question is rendered entirely on the Canvas with no text nodes in the DOM
☐	Attempting to copy the Canvas content via Ctrl+C or right-click yields no usable text in the clipboard
☐	The monospace code block renderer correctly displays a sample 20-line code snippet with syntax token colors
☐	The Web Worker renders the challenge without blocking the main thread (verified via browser performance profiler — no long tasks on main thread during challenge paint)
☐	The answer input zone captures keyboard input and displays typed characters on the Canvas
☐	The contextmenu event is suppressed during an active session
☐	Accessibility mode renders the challenge in DOM elements when enabled by the recruiter
☐	Accessibility mode activation is logged and visible in the session record

PHASE 10

Arcade Mode — Single Player Blitz (Timer Mode)

Lobby UI, Timer Mode flow, live session UI, round progression, results screen

Phase 10 assembles the first complete, end-to-end playable game mode: Single Player Blitz in Timer Mode. This phase connects the frontend, Game API, Question Engine, and Code Execution Engine into a single functional evaluation flow. A candidate can start a session, play through five rounds with a live timer, submit answers, and view their results.

Deliverables & Work Items

- **Lobby page — mode and format selection —** Implement the lobby page UI. Step 1: present Single Player vs Dual Mode cards. Step 2 (after Single Player selection): present Timer Mode vs Live Mode cards. Step 3 (after Timer Mode selection): present the category selection screen with cards for The Missing Link, The Bottleneck Breaker, and State Tracing. Each step uses animated transitions. The selected path is clearly summarized before the session starts.
- **Session initialization —** On session start, call POST /sessions on the Game API. Display a countdown (3-2-1) before the first round begins. Connect the WebSocket to the Game API session channel.
- **Active session UI —** Implement the game session page: top bar showing the round number (Round X of 5), time remaining bar (animated, color shifts from green to amber to red as time decreases), and score. Below: the Canvas challenge display fills the remaining viewport. Below the Canvas: the answer submission zone (either multiple choice buttons or the Canvas text input, depending on question type).
- **Timer bar animation —** The timer bar depletes smoothly from full to empty over the round time limit. The animation runs at 60fps via requestAnimationFrame. At 50% time remaining, the bar transitions to amber. At 25%, it transitions to red and a subtle pulse animation begins.
- **Round result display —** After answer submission or timeout, display an in-session result overlay on the Canvas: correct or incorrect, the correct answer explanation, points awarded, and time taken. After 3 seconds (or on user tap/click), advance to the next round.
- **Code submission flow —** For questions that require code execution: after the candidate finishes writing in the Canvas input zone, a Run & Submit button triggers a POST to the Code Execution Engine via the Game API proxy. Display an execution spinner on the Canvas while awaiting the result. Display the execution result (stdout, error, or correct output) inline on the Canvas.
- **Results page —** Implement the post-session results page: total score, time taken per round, correctness breakdown, a star rating (1-3 stars based on score), and a share prompt. The results page uses DOM rendering (not Canvas) since it is post-session and the anti-cheat Canvas requirement no longer applies.

Phase Completion Checklist

☐	A candidate can complete a full five-round Single Player Timer Mode session without error
☐	The timer bar depletes in real time and triggers a TIMEOUT round result on expiry
☐	A correct answer increments the score by the correct point value
☐	The round result overlay displays the correct answer explanation after each round
☐	A code submission is sent to the Code Execution Engine and the result is displayed on the Canvas within 3 seconds
☐	The results page displays the correct score, per-round breakdown, and star rating
☐	The WebSocket connection is closed cleanly when the session completes or the page is navigated away from
☐	No challenge text is present as a readable DOM text node during an active session

PHASE 11

Arcade Mode — Live Mode & Lives System

Live Mode lobby, randomized question dispatch, lives display, session-fail flow

Phase 11 extends the Blitz Mode implementation with Live Mode. Live Mode introduces the randomized question pool — including Syntax Error Detection questions exclusive to this mode — and the lives system that creates the continuous-pressure gameplay feel. Phase 11 also delivers the session-fail state and its corresponding UI.

Deliverables & Work Items

- **Live Mode lobby path —** Extend the lobby page to handle the Live Mode selection flow. After selecting Live Mode (no category selection step), display the lives allocation screen showing the 3-4 lives the candidate will have for the session. A prominent visual of the life count (pixel-art hearts or shields) is displayed before the session starts.
- **Randomized question dispatch integration —** Connect the Game API's Live Mode session type to the Question Engine's random dispatch endpoint. The Question Engine must select a category at random for each round from the full four-category pool including Syntax Error Detection. The category selected for each round is logged in the session record.
- **Lives display in active session UI —** Add a lives display to the top bar of the active session UI. Render lives as pixel-art icons. When a life is lost, play a loss animation: the icon breaks/dims and a brief screen flash occurs. The candidate always knows their current life count.
- **Life deduction logic —** A life is deducted on: timeout (TIMER_TICK exhaustion), incorrect answer submission, or fatal syntax error in a code submission (runtime exception that produces no output). Life deduction is determined by the Game API

based on the ROUND_COMPLETE event outcome. The Anti-Cheat Service is notified of each LIFE_LOST event for inclusion in the session record.

- **Session-fail state and UI —** When all lives are exhausted before round 5, the Game API transitions the session to SESSION_FAILED and broadcasts the event. The frontend displays a game-over screen: pixel-art game over animation, final score, rounds completed, lives lost breakdown. A retry button creates a new session. The session-fail result is stored with full round-by-round data for recruiter review.
- **Syntax Error Detection challenge rendering —** Implement Canvas rendering support for Syntax Error Detection challenges. The renderer must display a code block with one or more errors highlighted visually (underline or background highlight in error red). The candidate interacts with the Canvas to indicate which line(s) contain errors and what the correct syntax should be.

Phase Completion Checklist	
☐	A Live Mode session dispatches questions from all four categories across five rounds
☐	A Syntax Error Detection question is displayable in the Canvas renderer with error highlights
☐	A timeout deducts one life and broadcasts LIFE_LOST over WebSocket
☐	An incorrect answer deducts one life in Live Mode
☐	The lives display updates immediately after each LIFE_LOST event
☐	Exhausting all lives before round 5 transitions to SESSION_FAILED and displays the game-over screen
☐	The session-fail result is persisted and retrievable via GET /sessions/:id/result
☐	The category selected for each round is logged in the session record

PHASE 12

Arcade Mode — Dual Mode (Head-to-Head)

Matchmaking, synchronized sessions, opponent progress display, dual results

Phase 12 implements Dual Mode — the head-to-head competitive format where two candidates are matched and evaluated simultaneously on the same question set. Both candidates see the same questions (with independent semantic randomization applied), their own timer, and a live indicator of the opponent's progress. Dual Mode introduces matchmaking and synchronized session orchestration.

Deliverables & Work Items

- **Matchmaking queue —** Implement a matchmaking queue in the Game API. When a candidate selects Dual Mode, they enter the queue. When two candidates are in the queue for the same session type and category (Timer Mode) or session type alone (Live Mode), the Game API creates a paired session, assigns both candidates to it, and notifies both via WebSocket.
- **Paired session creation —** A paired session is a single session record with two participant slots. Both participants receive the same question sequence. Each participant has independent answer state, timing, and score. The Game API broadcasts ROUND_COMPLETE events to both participants simultaneously after both have submitted or timed out.
- **Opponent progress display —** In the active session UI for Dual Mode, add an opponent status panel. Display: the opponent's current round status (thinking, submitted, timed out), the opponent's score relative to the candidate's, and a subtle animation when the opponent submits. Do not display the opponent's answer or timing details during the round — only their submission status.
- **Round synchronization —** In Dual Mode, a round ends when both participants have submitted or when the timer expires — whichever occurs first. After one participant submits, the timer continues running for the other. If the timer expires, the slower participant's round is concluded by timeout. The next round does not begin until both participants' round states are ROUND_COMPLETE.
- **Dual Mode results page —** The results page for a Dual Mode session displays both candidates' results side by side: total score, per-round correctness, time-per-round, and a winner declaration. The winner is the candidate with the higher total score; ties are declared explicitly.
- **Matchmaking timeout —** If a queued candidate cannot be matched within 60 seconds, offer them: wait longer, switch to Single Player mode, or cancel. Implement a countdown display in the matchmaking waiting screen.

Phase Completion Checklist	
☐	Two candidates selecting Dual Mode Timer Mode with the same category are matched within the 60-second window
☐	Both matched candidates receive the same question sequence
☐	The opponent status panel updates in real time when the opponent submits
☐	A round does not advance until both participants' states are ROUND_COMPLETE
☐	The Dual Mode results page correctly declares the winner based on total score
☐	A tie is declared when both candidates achieve the same total score
☐	A queued candidate unmatched after 60 seconds is presented with the documented fallback options
☐	Both participants' session records are persisted with full round-by-round data

PHASE 13

Story Mode — The Archive (Database Normalization)

Campaign chapter UI, corrupted dataset challenge, normalization evaluation engine

Phase 13 begins Story Mode implementation with its first chapter: The Archive. Story Mode differs fundamentally from Arcade Mode — challenges are multi-step, narrative-driven, and evaluated on correctness of reasoning rather than speed alone. The Archive evaluates database normalization: candidates repair a corrupted dataset and decompose it into a correct schema.

Deliverables & Work Items

- **Story Mode chapter selection UI —** Implement the Story Mode entry screen. Display three chapter cards: The Archive (database), The Shield Generator (OOP), The Aether-Stream (networking). Each card shows the chapter name, a brief description, estimated completion time, and a lock/unlock state (chapters unlock sequentially — completing The Archive unlocks The Shield Generator).
- **Narrative introduction —** Each chapter opens with a short narrative introduction: a text panel with pixel-art scene illustration and a typewriter-effect story paragraph that sets the context. The Archive narrative frames the challenge as recovering data from a crashed legacy database. The introduction is skippable after 3 seconds.
- **Corrupted dataset display —** Render the initial corrupted dataset as a visual table on the Canvas. The table contains intentional normalization violations: repeating groups, partial dependencies, and transitive dependencies. Cells with integrity violations are highlighted visually. Candidates must interact with the Canvas to select and rearrange data into corrected tables.
- **Normalization interaction model —** Implement the interaction model: candidates drag columns from the corrupted table into new target tables rendered on the Canvas. They assign primary key columns (click to designate as PK), foreign key relationships (draw connector lines between tables), and table names (typed into Canvas input zones). The interaction must work with both mouse and keyboard.
- **Normalization evaluation engine —** After submission, evaluate the candidate's schema against the correct 3NF decomposition. The evaluator checks: all attributes are in the correct table, all primary keys are correctly identified, all foreign key relationships are correct, and the decomposition is lossless (can reproduce the original data via joins). Return a partial score for partially correct submissions — not binary pass/fail.
- **Chapter result and narrative resolution —** Display the chapter result: score, which normalization requirements were met, and a brief narrative resolution. If the chapter is completed above the pass threshold (70%), the next chapter is unlocked and the unlock animation plays.

Phase Completion Checklist	
☐	The chapter selection screen displays all three chapters with correct lock/unlock states

☐	The Archive narrative introduction plays with typewriter animation and is skippable
☐	The corrupted dataset is rendered on the Canvas with violation highlights visible
☐	A candidate can drag columns, assign keys, and draw foreign key relationships using the Canvas interaction model
☐	A submission with a fully correct 3NF schema receives a full score
☐	A submission with a partially correct schema receives a proportional partial score
☐	A submission above 70% unlocks The Shield Generator chapter
☐	The chapter result screen correctly displays which normalization requirements were and were not met

PHASE 14

Story Mode — The Shield Generator & The Aether-Stream

OOP/system design chapter, networking chapter, story completion flow

Phase 14 completes Story Mode by implementing the second and third chapters. The Shield Generator evaluates OOP and system design ability through a refactoring challenge. The Aether-Stream evaluates networking fundamentals through a packet ordering and validation challenge. Phase 14 also implements the story completion experience.

Deliverables & Work Items

- **The Shield Generator — tightly coupled system display —** Render the initial tightly coupled class hierarchy on the Canvas as a simplified UML-style diagram. Show coupling violations visually — excessive direct dependencies highlighted in red. The narrative frames this as a broken shield system that must be repaired before the station loses power.
- **The Shield Generator — refactoring interaction model —** Candidates interact with the Canvas diagram to: extract interfaces (select a class, click Extract Interface, name the interface in a Canvas input), apply Dependency Injection (drag a dependency arrow from a class to an interface rather than to a concrete class), and select the Strategy Pattern application (multiple-choice Canvas selection for which component should vary by strategy).
- **The Shield Generator — OOP evaluation engine —** Evaluate the candidate's refactored design: interfaces correctly extracted, dependency arrows correctly pointed at interfaces (not concrete classes), Strategy Pattern correctly applied, and coupling score reduced below the threshold. Score each criterion independently and return a breakdown.
- **The Aether-Stream — packet stream display —** Render the packet stream as a sequence of numbered data blocks on the Canvas. Blocks are rendered out of order. Each block displays its sequence number, payload preview, and checksum. Some

blocks have invalid checksums (corrupted), and some have flagged payloads (simulated malicious content). A time pressure element displays a countdown — the stream closes when the timer expires.

- **The Aether-Stream — interaction model —** Candidates drag packet blocks into a reorder zone, marking them in correct sequence order. They flag checksum-invalid blocks as corrupted (click a flag icon on the block). They mark payload-flagged blocks as malicious (different visual flag). Submission is triggered manually or automatically on timer expiry.
- **The Aether-Stream — networking evaluation engine —** Evaluate: packets in correct sequence order, all corrupted packets identified and only corrupted packets identified (penalize false positives), all malicious payloads identified and only malicious payloads identified. Score each criterion and return a breakdown.
- **Story Mode completion screen —** After all three chapters are completed, display the full story completion screen: total story score across all chapters, individual chapter grades, a completion badge (different tiers based on score), and a shareable result summary.

Phase Completion Checklist	
☐	The Shield Generator tightly coupled diagram renders with coupling violations highlighted
☐	An interface extraction interaction correctly creates a new interface node on the Canvas diagram
☐	A dependency arrow pointing to an interface (not a concrete class) is evaluated as correct DI
☐	The OOP evaluation engine correctly scores each of the four criteria independently
☐	The Aether-Stream renders out-of-order packets with sequence numbers and checksum values
☐	Dragging packets into the correct sequence order is reflected in the submission payload
☐	A flagged malicious payload correctly contributes to the malicious identification score
☐	Completing all three chapters above 70% each displays the story completion screen with the correct badge tier

PHASE 15

Recruiter Dashboard

Candidate results, anti-cheat flags, session replays, analytics overview

Phase 15 builds the recruiter-facing dashboard — the primary interface through which hiring teams consume LogicForge evaluation results. The dashboard surfaces session scores, risk

flags, behavioral telemetry summaries, and aggregate analytics. All dashboard pages use server-rendered Next.js components for fast initial load and data freshness.

Deliverables & Work Items

- **Candidate list view —** The dashboard home displays a paginated, sortable, filterable table of all candidates who have completed sessions associated with the recruiter's organization. Columns: candidate name, email, session mode, score, completion status, risk flag level, and session date. Filtering by flag level, mode, score range, and date range. Sorting by all columns.
- **Individual candidate session detail —** A detail page for each session shows: the complete round-by-round score breakdown, time-per-round, code submissions and their execution results, the category sequence (for Live Mode), and the full risk flag history with timestamps and contributing events.
- **Behavioral telemetry visualization —** On the session detail page, render a timeline visualization of telemetry events: focus loss/restore events shown as colored bands, keystroke cadence shown as a bar chart over time, solution times shown as data points against the difficulty baseline. This visualization is a DOM-based chart (not Canvas — it is post-session recruiter content, not candidate-facing).
- **Risk flag export —** Provide a button to export the session's telemetry data and risk flag report as a PDF or CSV. The export is generated server-side and downloaded as a file attachment. The export includes: all telemetry events, flag timestamps, risk score progression, and the recruiter's session configuration.
- **Organization analytics overview —** An analytics page shows aggregate metrics across all sessions for the organization: average score by mode and category, completion rate, flag rate (percentage of sessions with HIGH flags), average time-per-round, and score distribution histogram. All charts use accessible color palettes.
- **Live session monitoring —** For active Dual Mode sessions, the recruiter dashboard provides a live monitoring view: real-time risk scores for both participants, live telemetry event feed, and round progress. This view subscribes to the Anti-Cheat Service's recruiter WebSocket channel.

Phase Completion Checklist	
☐	The candidate list correctly displays all sessions associated with the recruiter's organization
☐	Filtering by HIGH flag level returns only sessions with HIGH risk scores
☐	The individual session detail page displays correct round-by-round scores
☐	The behavioral telemetry timeline renders focus loss events as colored bands at correct timestamps
☐	The risk flag export downloads a correctly formatted file containing all required fields
☐	The analytics overview displays correct aggregate metrics that match the underlying session data

☐	The live monitoring view displays the risk score updating in real time during an active session
☐	A recruiter from Organization A cannot view sessions from Organization B

PHASE 16

Testing, Security Hardening & Performance Validation

Integration tests, load testing, penetration testing, sub-100ms latency validation

Phase 16 is a dedicated quality and security phase. No new features are implemented. The goal is to verify that all platform functions work correctly under load, that the security boundaries hold under adversarial conditions, and that the latency targets documented in the NFRs are met. Every finding from this phase that is classified as critical or high severity must be resolved before Phase 17 begins.

Deliverables & Work Items

- **End-to-end integration tests —** Write integration tests that simulate complete user journeys: candidate registration through session completion through result retrieval, and recruiter login through dashboard access through flag export. These tests run against a fully-assembled Docker Compose environment and interact with all services.
- **Anti-cheat bypass testing —** Attempt to extract challenge content using: browser developer tools DOM inspection, clipboard access via JavaScript, browser extensions that intercept rendering, screenshot-based OCR on the Canvas output, and semantic LLM inference using question variants. Document all attempts and their outcomes. Verify that Canvas Obfuscation and Semantic Randomization defeat all tested approaches.
- **Code Execution Engine security testing —** Submit code designed to: escape the container sandbox via known Docker vulnerabilities, exhaust CPU or memory beyond configured limits, make outbound network connections, write to paths outside the scratch directory, and execute shell injection via the language runtime. All attempts must fail with appropriate error responses.
- **Load testing — Arcade Mode —** Simulate 100 concurrent Arcade Mode sessions. Measure: WebSocket timer tick delivery latency (target: median below 50ms, 99th percentile below 100ms), answer submission round-trip latency (target: below 300ms), and code execution latency (target: below 3 seconds for correct Python submissions). All targets must be met at 100 concurrent sessions.
- **Authentication security testing —** Test: JWT signature forgery (must be rejected), token replay after logout (must be rejected), refresh token reuse after rotation (must be rejected), role escalation via modified token payload (must be rejected), and brute-force login rate limiting (must trigger after 10 attempts in 60 seconds).
- **Data isolation testing —** Verify that: Candidate A cannot retrieve Candidate B's session results, Recruiter A cannot retrieve Recruiter B's candidates, a valid candidate session token cannot access recruiter dashboard endpoints, and the audit log cannot be modified via any API endpoint.

Phase Completion Checklist	
☐	All end-to-end integration tests pass in the Docker Compose environment
☐	All attempted Canvas content extraction methods yield no usable challenge text
☐	All Code Execution Engine sandbox escape attempts fail with appropriate error responses
☐	Arcade Mode WebSocket timer tick latency is below 100ms at the 99th percentile under 100 concurrent sessions
☐	Code execution latency for correct Python submissions is below 3 seconds under load
☐	JWT signature forgery and token replay attacks are rejected with 401 responses
☐	The audit log has no API endpoints that permit modification of existing records
☐	Cross-organization data isolation is confirmed — no session from Organization A is accessible to Organization B
☐	All critical and high severity security findings are resolved and re-tested

PHASE 17

Production Deployment, Observability & Launch Readiness

CI/CD pipeline, production Docker configuration, logging, monitoring, alerting, launch checklist

Phase 17 prepares LogicForge for production deployment. This phase delivers the CI/CD pipeline, production-hardened Docker configurations, structured observability infrastructure (logging, metrics, alerting), and a launch readiness checklist that must be fully satisfied before the platform is opened to external users.

Deliverables & Work Items

- **CI/CD pipeline —** Configure a CI/CD pipeline (GitHub Actions or equivalent) with stages: install, lint, type-check, unit-test, integration-test, build, push-container, deploy. Use Turborepo's remote cache in CI to eliminate redundant builds. The pipeline must run on every pull request and every merge to main. Deployment to production is gated on a manual approval step after all tests pass.
- **Production Docker Compose configuration —** Create docker-compose.prod.yml with: resource limits (CPU and memory) for each service, restart policies (always), named volumes for persistent data, no exposed ports except the frontend and API gateway, and secrets management via Docker Secrets or environment injection (never hardcoded values).

- **API gateway and TLS termination —** Configure an API gateway (nginx or Caddy) to: terminate TLS with valid certificates, route /api/* to backend services, proxy WebSocket upgrades correctly, enforce HTTP Strict Transport Security headers, and rate-limit per IP.
- **Structured logging pipeline —** Verify that all services emit JSON logs via @logicforge/logger. Configure a log aggregation destination (e.g. a log management service). All logs must include correlation IDs enabling a single candidate request to be traced across all services. Implement log retention policy: 90 days for standard logs, 1 year for audit logs.
- **Metrics and alerting —** Instrument each service with health metrics: request rate, error rate, and latency percentiles (p50, p95, p99). Configure alerts: error rate above 1% for more than 5 minutes, p99 latency above 500ms for more than 5 minutes, any service container restart. Alerts must deliver to a documented on-call channel.
- **GDPR compliance verification —** Verify that: all telemetry data is associated with explicit consent (collected at registration), a data deletion endpoint exists that removes all personal data for a given user_id, data export is available on request, and all data processing activities are documented in a processing record.
- **Launch readiness checklist execution —** Complete and sign off the full launch readiness checklist (see Phase Completion Checklist below). Every item must have an assigned owner and a verification method. No item may be marked complete without the verification method having been executed.

Phase Completion Checklist	
☐	The CI/CD pipeline runs lint, type-check, unit tests, and integration tests on every pull request
☐	A failed test blocks merging to main — the pipeline enforces this and cannot be bypassed without documented exception
☐	The production Docker Compose starts all services with resource limits and restart policies configured
☐	TLS is terminated at the API gateway and all HTTP traffic is redirected to HTTPS
☐	A single candidate session can be traced across all service logs using a correlation ID
☐	Alert firing is verified by intentionally triggering a high error rate in a staging environment
☐	The data deletion endpoint removes all personal data for a test user and is verified via database inspection
☐	All Phase 17 launch readiness checklist items are signed off by their assigned owners
☐	The platform is accessible over HTTPS at the production domain with valid TLS certificates
☐	A smoke test (registration, login, complete one Arcade session, view results, logout) passes in the production environment